

Zwischenbericht — Campus Next-Gen Data-Hub

Evaluation von Airbyte als ETL-/Integrationswerkzeug (Ablösung von Talend)

Modul	INF-M Modul Projekte SoSe '26
Gruppe	Airbyte
Bearbeiter	Lahres Timo tlahres@stud.hs-offenburg.de Bräutigam Rebecca rbraeuti@stud.hs-offenburg.de Horst Isabella ihorst@stud.hs-offenburg.de
Stand	07.06.2026
Abgabe	07.06.2026 (Zwischenbericht + Doku)
GitHub	https://github.com/Timbo3399/INFM_Airbyte

1. Projektziel & Kontext

Im Projekt evaluieren wir [Airbyte](#) als modernes ETL-/Datenintegrationswerkzeug mit dem Ziel, das bisher eingesetzte **Talend** in der Hochschul-IT abzulösen. Die gesamte Evaluationsumgebung läuft **lokal in Docker Desktop** und bildet einen realistischen Ausschnitt der Hochschul-Datenlandschaft (anonymisierte Studierenden-, Gebäude-, Instituts- und Personaldaten) ab.

Die Evaluation ist in **sechs Testszenarien** gegliedert (DB-Connector, Facility-Management-Sync, Bild-BLOBs, Studenten/Personal-Mapping, Incremental Sync/IdM, Web-APIs) — siehe [testszenarien.md](#).

2. Erreichte Meilensteine (Stand 06.06.2026)

Meilenstein (lt. Betreuer-Mail)	Status	Beleg / Doku
Installation des Systems	✅ DB-Stack + Airbyte (abctl) lauffähig	installation-guide.md
Zugang für Betreuer	🕒 dokumentiert, Einrichtung im Termin	zugang.md
Einfacher ETL-Prozess	✅ durchgeführt & verifiziert (Postgres→Postgres)	etl-prozess.md
Beginn der Dokumentation	✅ README + 8 Dokumente unter docs/	dieses Repo

2.1 Installation

Das Setup ist vollständig skriptbasiert und reproduzierbar:

- [scripts/install.ps1](#) (Windows) bzw. [scripts/install.sh](#) (Linux/macOS) startet den kompletten Datenbank-Stack (Source-PostgreSQL, Ziel-PostgreSQL, Ziel-MySQL, CSV-File-Server), wartet auf den

`healthy`-Status und lädt die Testdaten.

- `scripts/setup-airbyte.ps1` / `scripts/setup-airbyte.sh` installiert Airbyte Community Edition über das offizielle CLI `abctl` in einem lokalen Kubernetes-Cluster (kind) innerhalb von Docker Desktop.
- **Plattformen:** Das Setup steht für **Windows, Linux und macOS** bereit (identische Logik, plattformspezifische Skripte).

Alle vier Datenbank-/Server-Container laufen verifiziert im Zustand `healthy`.

2.2 Datenbasis (Source-PostgreSQL)

Die anonymisierten Testdaten sind geladen:

Tabelle	Zeilen	Lademechanismus
<code>fm_gebaeude</code>	25	<code>scripts/load_fm_gebaeude.py</code>
<code>fm_inst</code>	2.083	<code>scripts/load_fm_inst.py</code>
<code>k_plz</code>	34.172	<code>scripts/load_k_plz.py</code>
<code>fm_rna</code>	379	<code>scripts/load_json.py</code>
<code>hso_personal</code>	870	<code>scripts/load_json.py</code>
<code>hso_students</code>	0	⚠ nur via File-Connector (s. Kap. 4)
<code>fm_stamm</code>	0	keine Quelldatei (s. Kap. 5)

2.3 Angelegte Airbyte-Connectoren & erster ETL-Lauf

In Airbyte sind angelegt und per Verbindungstest grün:

- **Sources (5):** `H50 Source PostgreSQL` (Postgres, Update-Methode *User Defined Cursor*), sowie vier File-Connectoren (`local`, `/local/*.csv`): `H50 CSV hso_students`, `H50 CSV k_plz`, `H50 CSV fm_gebaeude`, `H50 CSV fm_inst`.
- **Destinations (2):** `H50 Dest PostgreSQL` (Port 5434), `H50 Dest MySQL` (Port 3306, SSL aus, `allowPublicKeyRetrieval=true`, Raw-DB `destdb`).

Es wurden **drei Connections** (jeweils *Full Refresh* | *Overwrite*) ausgeführt und das Ergebnis **unabhängig in der jeweiligen Ziel-DB** geprüft:

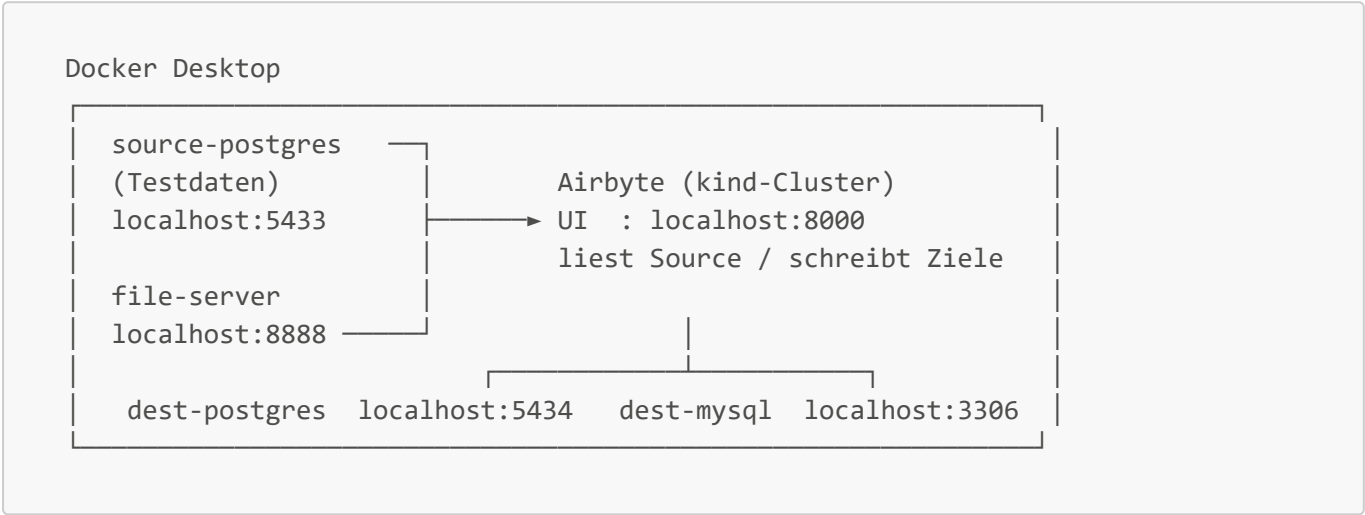
Connection	Streams	Ergebnis (Ziel-DB)
<code>H50 Source PostgreSQL → H50 Dest PostgreSQL</code>	<code>fm_gebaeude</code> , <code>k_plz</code>	25 / 34.172 ✓
<code>H50 Source PostgreSQL → H50 Dest MySQL</code>	<code>fm_gebaeude</code> , <code>k_plz</code>	25 / 34.172 ✓
<code>H50 CSV hso_students → H50 Dest PostgreSQL</code>	<code>hso_students</code> (File)	5.052 Zeilen ✓

Bemerkenswert: Der **File-Connector lud die defekte `hso_students.csv` vollständig (5.052 Zeilen)** in die DB — dieselbe Datei, an der ein direktes PostgreSQL-COPY scheiterte (0 Zeilen, Kap. 5.2). Pandas im File-Connector toleriert die Spalten- Inkonsistenzen. Damit sind **DB-Connector** (PG→PG, PG→MySQL) **und File-Connector** (CSV→DB) — also der Kern von Szenario 1 inkl. „PostgreSQL→MySQL dumpen“ — nachgewiesen.

Zusätzlich stellt der Dienst **PostgREST** (`hso_postgrest`, Szenario 6a) eine REST-API auf die Ziel-DB bereit: `GET http://localhost:3000/k_plz?limit=5` liefert die synchronisierten Daten als JSON.

3. Architektur (Kurzüberblick)

Detaillierte Beschreibung in `architektur.md`. Kern: alle Dienste laufen in Docker Desktop in einem gemeinsamen Netzwerk (`airbyte_net`). Airbyte selbst läuft in einem kind-Cluster und erreicht die Datenbanken über `host.docker.internal`.



4. Besonderheit Datenqualität der Quell-CSVs

Die bereitgestellten CSV-Dateien ließen sich **nicht** per direktem PostgreSQL-COPY laden (Details in Kap. 5). Wir haben daher tolerante Python-Loader implementiert, die die Daten nach dem Containerstart bereinigt einspielen. Eine Datei — `hso_students.csv` — ist strukturell so inkonsistent, dass sie aktuell nicht zuverlässig in die relationale Source-DB geladen werden kann; Studierendendaten werden stattdessen über den **Airbyte File-Connector** als Flatfile-Quelle eingebunden.

5. Probleme & Lösungen

5.1 Windows-/Umgebungsspezifische Hürden

Problem	Ursache	Lösung
<code>install.ps1</code> meldete „Python gefunden“, JSON-Laden schlug aber fehl	<code>python</code> ist unter Windows oft nur der Microsoft-Store-Platzhalter ; <code>Get-Command</code> findet ihn, er liefert aber keine echte Version	Echte Versionsprüfung (<code>py/python/python3</code>); zusätzlich Docker-Fallback , der die Daten ganz ohne Host-Python lädt
<code>setup-airbyte.ps1</code> fand kein <code>abctl</code> -Asset	Falsches Namensschema (<code>abctl_Windows_amd64.zip</code>) — korrekt ist <code>abctl-<version>-windows-<arch>.zip</code> ; zudem liegt <code>abctl.exe</code> in einem Unterordner	Asset per Muster ermitteln, aus Unterordner entpacken; Architektur-Erkennung PowerShell-7-fest

Problem	Ursache	Lösung
File-Server-Container dauerhaft unhealthy , Setup lief in Timeout	Healthcheck nutzte localhost → im Container zuerst IPv6 (:::1), nginx lauscht aber nur auf IPv4	Healthcheck auf 127.0.0.1 umgestellt

5.2 Datenqualität der Quell-CSVs (Hauptproblem)

Der SQL-Init lud die drei Kern-Tabellen **gar nicht** — **COPY** brach unter **ON_ERROR_STOP** bereits an der ersten fehlerhaften Datei ab, wodurch alle folgenden leer blieben. Die konkreten Probleme:

- **fm_gebaeude.csv**: unquotierte Kommas in Textfeldern (z. B. „Hörsäle,Bib.,RZ“), eingebettete Header-Zeilen.
- **k_plz.csv**: **3.417 Header-Zeilen** mitten in den Daten (zusammengesetzte Einzel-Exporte), Zeilen mit zu wenig Feldern, ein Feld (**krskfz**) breiter als das Schema (Kreis-Namen).
- **fm_inst.csv**: 86 Spalten (Tabelle nutzt 24), semikolon-getrennt, vereinzelt **NUL-Bytes**, doppelt-kodierte UTF-8-Umlaute.
- **hso_students.csv**: **strukturell defekt** — Datenzeilen haben mehr Spalten (~50–56) als der eigene Header (40), dazu unbalanciertes Quoting und Float-formatierte Integer.

Lösung: tolerante Python-Loader (**scripts/load_*.py**), die Header filtern, fehlerhafte Zeilen reparieren bzw. überspringen, NUL-Bytes und Mojibake bereinigen und Typkonvertierungen best-effort vornehmen. Der SQL-**COPY**-Schritt wurde entfernt (**01_load_data.sql**).

5.3 Airbyte/abctl-spezifische Hürden

Airbyte Community läuft über **abctl** in einem **kind-Kubernetes-Cluster**. Daraus ergaben sich mehrere nicht-offensichtliche, in der offiziellen Doku nicht beschriebene Hürden, die wir analysiert und gelöst haben:

Problem	Ursache	Lösung
File-Connector (local) findet /local/*.csv nicht	Connector-Pods (kind) sehen das Docker-Volume oss_local_root nicht	CSV-Verzeichnis beim Install via abctl local install --volume "...:/local" direkt in den Cluster mounten
--volume mit Windows-Pfad: is not a valid volume spec	abctl trennt den Volume-String stur an : , der Laufwerks-Doppelpunkt (C:) kollidiert	Pfad in MSYS-Form /c/Users/... angeben
Alle Connector-Tests/Syncs hängen Pending	Aktiviertes lokales Volume erwartet PVC airbyte-local-pvc in jedem Job-Pod; es existierte nicht (persistentvolumeclaim not found)	PV (hostPath /local) + PVC airbyte-local-pvc anlegen; JOB_KUBE_LOCAL_VOLUME_ENABLED=true + Neustart von launcher/worker
abctl local credentials --	Kombinierter Aufruf löst einen Org-Lookup aus, der HTML statt	E-Mail und Passwort in zwei getrennten Aufrufen setzen (erst --email , dann --

Problem	Ursache	Lösung
email ... -- password ... schlägt fehl (unable to determine organization email, invalid character '<')	JSON liefert	password)
Connector-Auswahl heißt „ Postgres “ (nicht „PostgreSQL“); — Default-Update- Methode ist CDC		In der UI „Postgres“ wählen; Update-Methode auf <i>User Defined Cursor</i> stellen (CDC bräuchte wal_level=logical)

Alle Lösungen sind in [scripts/setup-airbyte.ps1/.sh](#) automatisiert und in [airbyte-setup.md](#) / [etl-prozess.md](#) dokumentiert.

6. Offene Punkte / Fragen an die Betreuer

1. **hso_students.csv Soll-Struktur?** Die Datei ist nicht eindeutig ladbar (mehr Datenspalten als Header, defektes Quoting). Ist es gewollt eine defekte Datei einzulesen, oder wird eine korrekt lesbare Datei bereitgestellt? Dies blockiert aktuell Szenario 4 (Account-Mapping in die Source-DB).
2. **fm_stamm** (Raumstammdaten): Es liegt keine Quelldatei vor. Wird diese Tabelle benötigt, und woher sollen die Daten kommen?
3. **Zugang für Betreuer:** Airbyte Community Edition ist im Wesentlichen Single-User und läuft auf **localhost**. Wie soll Ihr Zugang erfolgen? Gemeinsamer Admin-Login während des Vor-Ort-Termins, oder ist ein Remote-Zugang (z. B. Tunnel/VM) gewünscht? (Vorschlag in [zugang.md](#).)
4. **Sync-Strategie:** Wir nutzen den Cursor-Modus (**updatedat**) statt CDC, da CDC zusätzliche PostgreSQL-Konfiguration (logical WAL, Replication Slot) erfordert. Ist CDC für die Evaluation gewünscht?
5. **Scope:** Welche der sechs Szenarien haben für die Bewertung Priorität?

7. Anforderungs- und Szenarien-Status

Eine vollständige Gegenüberstellung aller Kickoff-Anforderungen und der sechs Szenarien mit Bewertung der Airbyte-Eignung und unserem Umsetzungsstand steht in [anforderungen.md](#). Kernbefunde:

- **Erfüllt:** Open Source/Community, PostgreSQL- & MySQL-Anbindung, CSV/JSON/Excel-Dateien, Logging/Monitoring, einfacher ETL-Lauf (verifiziert).
- **Einschränkungen ggü. Talend:** **kein** OSS-Connector für **Informix**; **kein XML** nativ; **keine freie Code-Snippet-Ausführung** (Mapping nur via dbt-SQL/Custom-Connector). Diese Punkte sind die zentralen Evaluationsbefunde.

8. Nächste Schritte

- Szenario 1 abrunden: Sync auch nach MySQL + ein File→DB-Sync (Nachweis File-Connector-Last).

- Weitere Szenarien: FM-Denormalisierung (dbt/View), BLOB-Bilder, Incremental Sync (IdM), Web-APIs (PostgREST/SOAP).
 - Klärung der offenen Fragen aus Kap. 6.
-

Anhang: Repository-Struktur

Siehe [README.md](#). Setup in unter 20 Minuten via `scripts/install.ps1` + `scripts/setup-airbyte.ps1`.